



University of Antwerp
| Faculty of Science

Computer Networks (2025-2026)

Part 1: Computer Networks and the Internet

Jeroen Famaey

jeroen.famaey@uantwerpen.be

Table of contents

1. History of the Internet
2. Internet architecture
3. Network performance
4. Protocol layers

History of the Internet

Link to the book
shown at start of
each section

1961 – Kleinrock theoretically proves effectiveness of packet-switching

- At the time, telephone networks made use of **circuit switching**, which works well for constant rate voice
- A solution was needed for transmitting computer data, which is more bursty



Mark J. Terrell/AP Photo



University of Antwerp
Faculty of Science

early packet switch



Leonard Kleinrock (MIT)

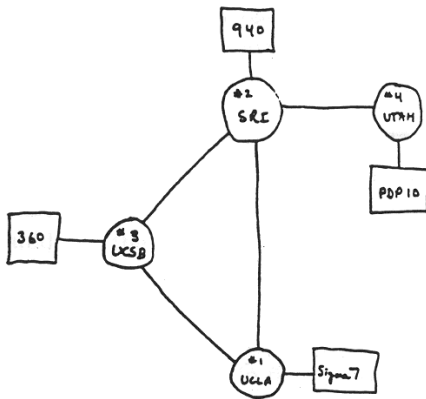
4

The field of computer networking and today's Internet trace their beginnings back to the early 1960s, when the telephone network was the world's dominant communication network. Telephone network uses circuit switching to transmit information from a sender to a receiver—an appropriate choice given that voice is transmitted at a constant rate between sender and receiver. Given the increasing importance of computers in the early 1960s and the advent of timeshared computers, it was perhaps natural to consider how to hook computers together so that they could be shared among geographically distributed users. The traffic generated by such users was likely to be *bursty*—intervals of activity, such as the sending of a command to a remote computer, followed by periods of inactivity while waiting for a reply or while contemplating the received response.

Three research groups around the world, each unaware of the others' work, began inventing packet switching as an efficient and robust alternative to circuit switching. The first published work on packet-switching techniques was that of Leonard Kleinrock, then a graduate student at MIT. Using queuing theory, Kleinrock's work elegantly demonstrated the effectiveness of the packet-switching approach for bursty traffic sources. In 1964, Paul Baran at the Rand Institute had begun investigating the use of packet switching for secure voice over military networks, and at the National Physical Laboratory in England, Donald Davies and Roger Scantlebury were also developing their ideas on packet switching.

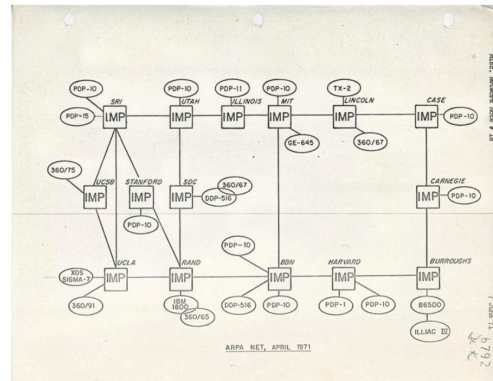
1967 – ARPAnet is conceived, the predecessor of the modern Internet

1969 (4 nodes)



THE ARPA NETWORK

1971 (15 nodes)



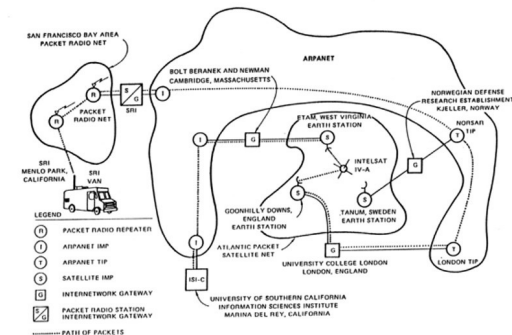
- First host-to-host protocol: Network Control Protocol (NCP)
- NCP enables applications (e.g., e-mail)

The work at MIT, Rand, and the NPL laid the foundations for today's Internet. But the Internet also has a long history of a let's-build-it-and-demonstrate-it attitude that also dates back to the 1960s. J. C. R. Licklider and Lawrence Roberts, both colleagues of Kleinrock's at MIT, went on to lead the computer science program at the Advanced Research Projects Agency (ARPA) in the United States. Roberts published an overall plan for the ARPAnet, the first packet-switched computer network and a direct ancestor of today's public Internet. On Labor Day in 1969, the first packet switch was installed at UCLA under Kleinrock's supervision, and three additional packet switches were installed shortly thereafter at the Stanford Research Institute (SRI), UC Santa Barbara, and the University of Utah. The fledgling precursor to the Internet was four nodes large by the end of 1969. Kleinrock recalls the very first use of the network to perform a remote login from UCLA to SRI, crashing the system.

By 1972, ARPAnet had grown to approximately 15 nodes and was given its first public demonstration by Robert Kahn. The first host-to-host protocol between ARPAnet end systems, known as the network-control protocol (NCP), was completed. With an end-to-end protocol available, applications could now be written. Ray Tomlinson wrote the first e-mail program in 1972.

1974 – Network of networks proposed by Vint Cerf and Robert Kahn

- In the early and mid 1970's many packet switched networks, besides ARPAnet, were deployed
- In 1974, Cerf and Kahn proposed an architecture to interconnect them ("internetworking")
- Their proposal includes an early version of TCP (encompassing the modern TCP and IP protocols)
- On **January 1, 1983** all hosts connected to ARPAnet were required to transition from NCP to TCP/IP



First Internet demonstration
(ARPAnet, PRNET, SATNET) in 1977



Vint Cerf



Robert Kahn

The initial ARPAnet was a single, closed network. In order to communicate with an ARPAnet host, one had to be actually attached to another ARPAnet IMP. In the early to mid-1970s, additional stand-alone packet-switching networks besides ARPAnet came into being: ALOHANet, a microwave network linking universities on the Hawaiian islands, as well as DARPA's packet-satellite and packet-radio networks; Telenet, a BBN commercial packet-switching network based on ARPAnet technology; Cyclades, a French packet-switching network pioneered by Louis Pouzin; Time-sharing networks such as Tymnet and the GE Information Services network, among others, in the late 1960s and early 1970s; IBM's SNA (1969–1974), which paralleled the ARPAnet work.

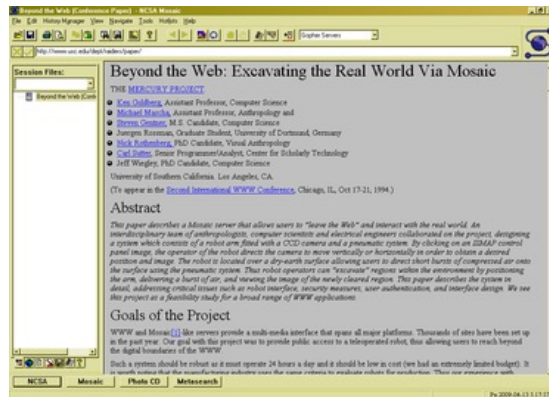
The number of networks was growing. With perfect hindsight we can see that the time was ripe for developing an encompassing architecture for connecting networks together. Pioneering work on interconnecting networks (under the sponsorship of the Defense Advanced Research Projects Agency (DARPA)), in essence creating a *network of networks*, was done by Vinton Cerf and Robert Kahn; the term *internetworking* was coined to describe this work. These architectural principles were embodied in TCP. The early versions of TCP, however, were quite different from today's TCP. The early versions of TCP combined a reliable in-sequence delivery of data via end-system retransmission (still part of today's TCP) with forwarding functions (which today are performed by IP). Early experimentation with TCP, combined with the recognition of the importance of an unreliable, non-flow-controlled, end-to-end transport service for applications such as packetized voice, led to the separation of IP out of TCP and the development of the UDP protocol. The three key Internet protocols that we see today—TCP, UDP, and IP—were conceptually in place by the end of the 1970s.

1991 – Creation of the World Wide Web (WWW) at CERN

- Between 1989 and 1991 Tim Berners Lee developed HTML, HTTP and an early browser and web server
- Belgian computer scientist Robert Cailliau contributed in the early stages



Robert Cailliau, Jean-François Abramatic and Tim Berners-Lee



Mosaic web browser

The main event of the 1990s was to be the emergence of the World Wide Web application, which brought the Internet into the homes and businesses of millions of people worldwide. The Web served as a platform for enabling and deploying hundreds of new applications that we take for granted today, including search (e.g., Google and Bing) Internet commerce (e.g., Amazon and eBay) and social networks (e.g., Facebook).

The Web was invented at CERN by Tim Berners-Lee between 1989 and 1991, based on ideas originating in earlier work on hypertext from the 1940s by Vannevar Bush and since the 1960s by Ted Nelson. Berners-Lee and his associates developed initial versions of HTML, HTTP, a Web server, and a browser—the four key components of the Web. Around the end of 1993 there were about two hundred Web servers in operation, this collection of servers being just a harbinger of what was about to come. At about this time several researchers were developing Web browsers with GUI interfaces, including Marc Andreessen, who along with Jim Clark, formed Mosaic Communications, which later became Netscape Communications Corporation. By 1995, university students were using Netscape browsers to surf the Web on a daily basis. At about this time companies—big and small—began to operate Web servers and transact commerce over the Web. In 1996, Microsoft started to make browsers, which started the browser war between Netscape and Microsoft, which Microsoft won a few years later.

1990-2000 – An explosion of “killer” applications on the Internet



The Web



Email



P2P filesharing



Instant
messaging

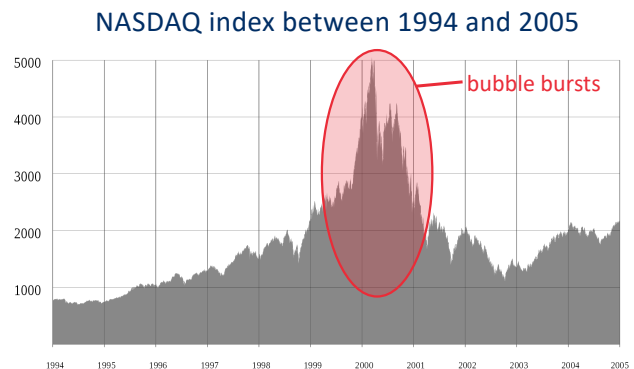
The second half of the 1990s was a period of tremendous growth and innovation for the Internet, with major corporations and thousands of start-ups creating Internet products and services. By the end of the millennium the Internet was supporting hundreds of popular applications, including four killer applications:

- E-mail, including attachments and Web-accessible e-mail
- The Web, including Web browsing and Internet commerce
- Instant messaging, with contact lists
- Peer-to-peer file sharing of MP3s, pioneered by Napster

Interestingly, the first two killer applications came from the research community, whereas the last two were created by a few young entrepreneurs.

2001 – Dotcom bubble bursts

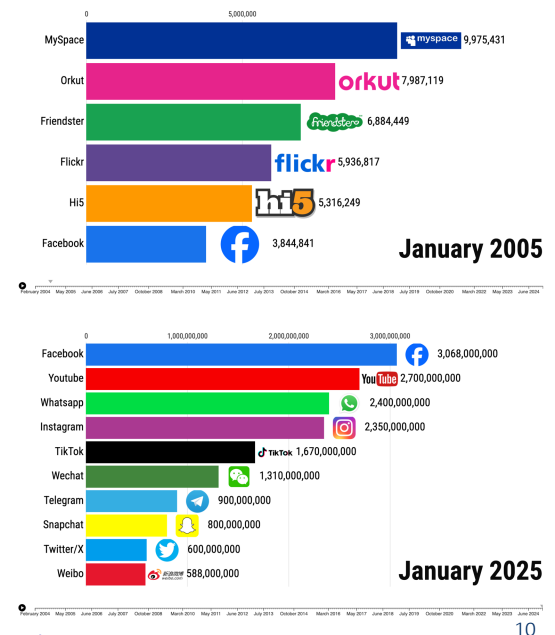
- 1000s of Internet start-ups made public offerings
- Valued in the billions without revenue streams
- Stocks collapsed in 2000-2001
- A few “big winners” survived the crash
 - Microsoft
 - Cisco
 - Yahoo
 - eBay
 - Google
 - Amazon
 - ...



The period from 1995 to 2001 was a roller-coaster ride for the Internet in the financial markets. Before they were even profitable, hundreds of Internet startups made initial public offerings and started to be traded in a stock market. Many companies were valued in the billions of dollars without having any significant revenue streams. The Internet stocks collapsed in 2000–2001, and many startups shut down. Nevertheless, a number of companies emerged as big winners in the Internet space, including Microsoft, Cisco, Yahoo, eBay, Google, and Amazon.

2000s – Birth of the interactive and social web

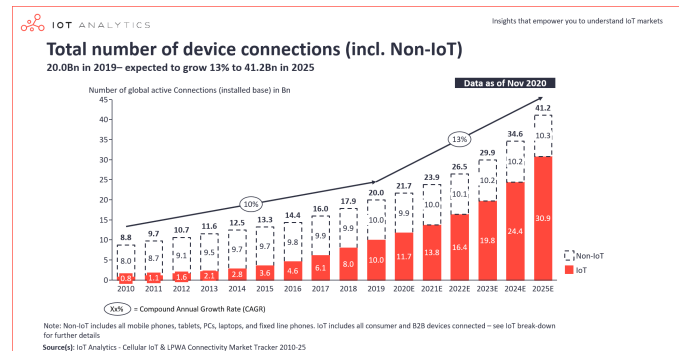
- **2003:** Launch of MySpace
 - Becomes most visited website on the web by 2005
 - Reaches peak of 115 million unique users in 2008
- **2006:** Facebook becomes accessible to the general public
 - Surpasses MySpace as most visited website by 2008
 - 430 million monthly active users in 2010
- Explosion of social networking websites and apps



Online social networks—such as Facebook, Instagram, Twitter, and WeChat (hugely popular in China)—have created massive people networks on top of the Internet. Many of these social networks are extensively used for messaging as well as photo sharing. Many Internet users today “live” primarily within one or more social networks. Through their APIs, the online social networks create platforms for new networked applications, including mobile payments and distributed games.

2010s – The Internet of Things (IoT)

- Connecting not only people, but also “things” to the Internet
- Early experiments with Internet-connected devices in the 1990s
 - 1990: “Internet Toaster”
 - 1991: “Trojan Room Coffee Pot”
- In the **2010s** everything became connected to the Internet

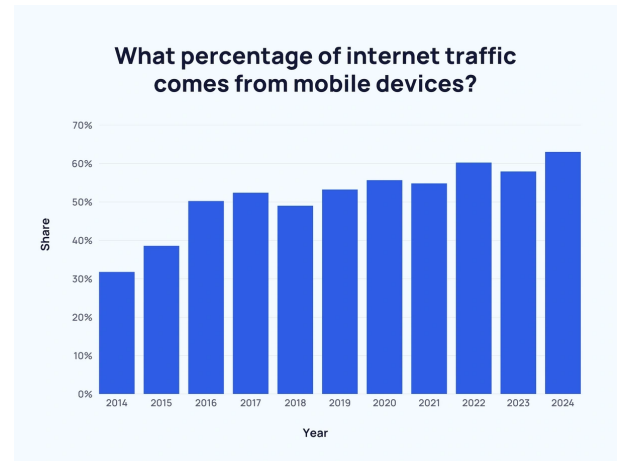


2010s – The mobile Internet era

- **2007:** Launch of the first iPhone
- Focus on mobile devices and experiences



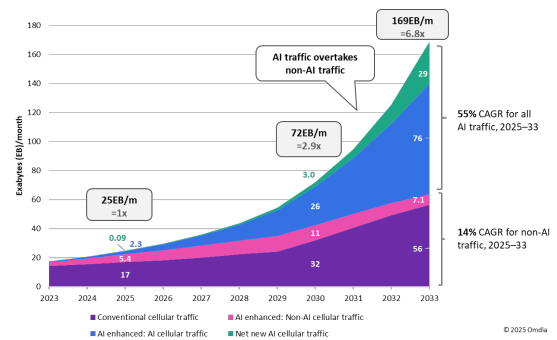
- Users can access the Internet anywhere at any time
- Widespread use of Internet streaming applications



Source: <https://explodingtopics.com/blog/mobile-internet-traffic>

2020s – AI-native web

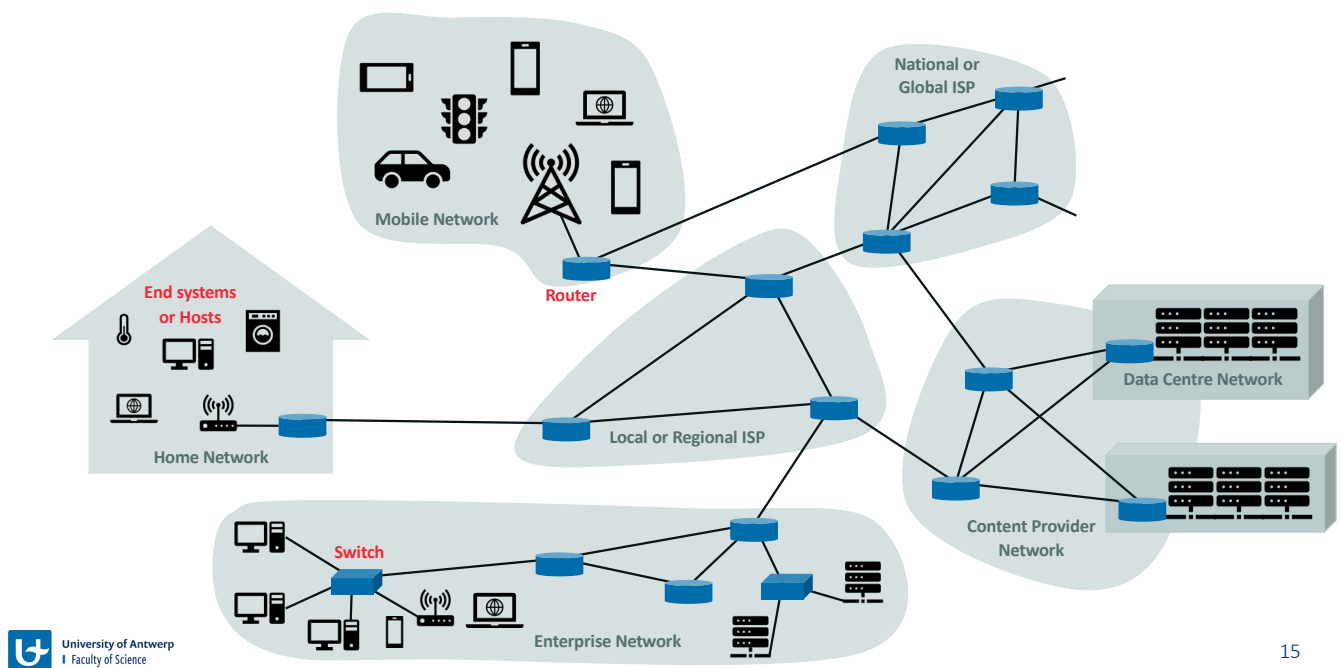
- Shift in how the Internet is being used
 - Before: Search → Click → Read → Decide
 - Now: Ask AI → Synthesized answer → Take action
- **2022**: Launch of ChatGPT to the public
- **2025**: Wide availability of Cloud-based AI services
- Possible **risks** for the future of the web / Internet
 - Flooding of the web with AI-generated content (e.g., “Dead Internet” theory)
 - Unauthorized use of (personal) data for AI model training
 - Need for massive compute and network resources (and energy), raising questions about sustainability



Source: <https://omdia.tech.informa.com/om135791/ai-network-traffic-report-update-for-europe-projected-cellular-growth-will-stress-access-infrastructure>

Internet Architecture

A high-level view of the Internet's architecture



15

End systems are connected together by a network of communication links and packet switches. Different links can transmit data at different rates, with the transmission rate of a link measured in bits/second. When one end system has data to send to another end system, the sending end system segments the data and adds header bytes to each segment. The resulting packages of information, known as packets in the jargon of computer networks, are then sent through the network to the destination end system, where they are reassembled into the original data.

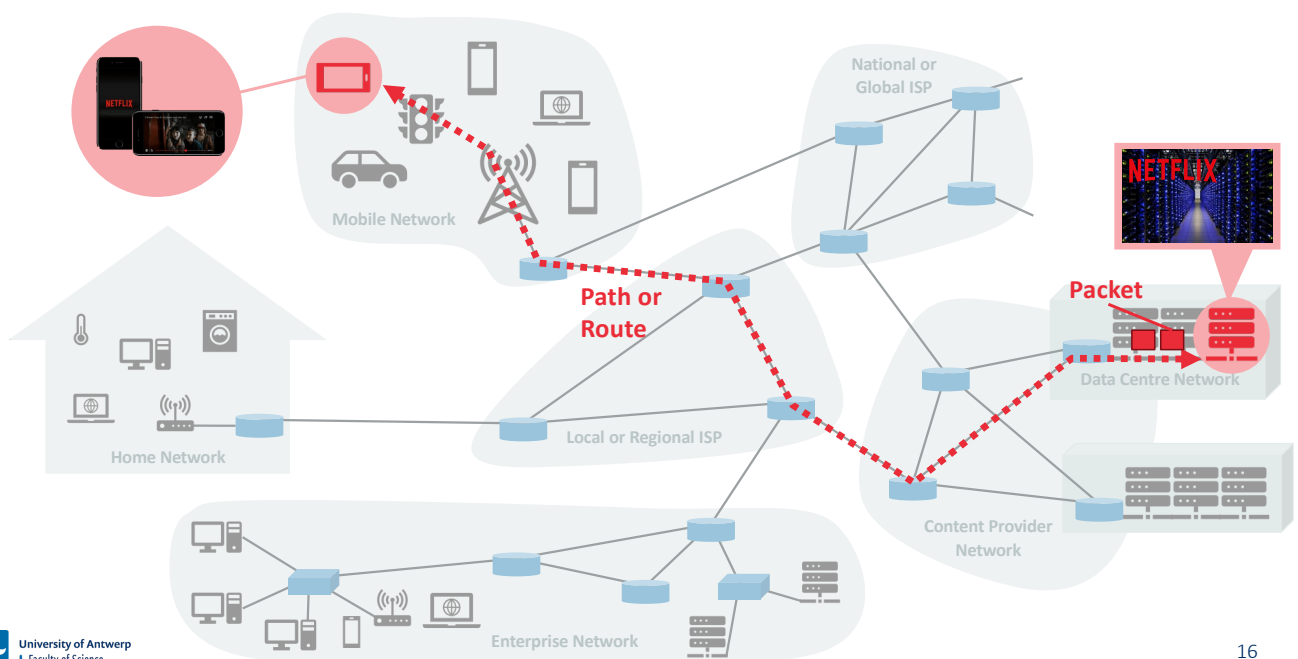
A packet switch takes a packet arriving on one of its incoming communication links and forwards that packet on one of its outgoing communication links. Packet switches come in many shapes and flavors, but the two most prominent types in today's Internet are routers and link-layer switches. Both types of switches forward packets toward their ultimate destinations. Link-layer switches are typically used in access networks, while routers are typically used in the network core. The sequence of communication links and packet switches traversed by a packet from the sending end system to the receiving end system is known as a route or path through the network.

End systems access the Internet through Internet Service Providers (ISPs), including residential ISPs such as local cable or telephone companies; corporate ISPs; university ISPs; ISPs that provide WiFi access in airports, hotels, coffee shops, and other public places; and cellular data ISPs, providing mobile access to our smartphones and other devices. Each ISP is in itself a network of packet switches and communication links.

The Internet is all about connecting end systems to each other, so the ISPs that provide

access to end systems must also be interconnected. These lower tier ISPs are thus interconnected through national and international upper-tier ISPs and these upper-tier ISPs are connected directly to each other. An upper-tier ISP consists of high-speed routers interconnected with high-speed fiber-optic links. Each ISP network, whether upper-tier or lower-tier, is managed independently, runs the IP protocol, and conforms to certain naming and address conventions.

Example: Watching movies online



16

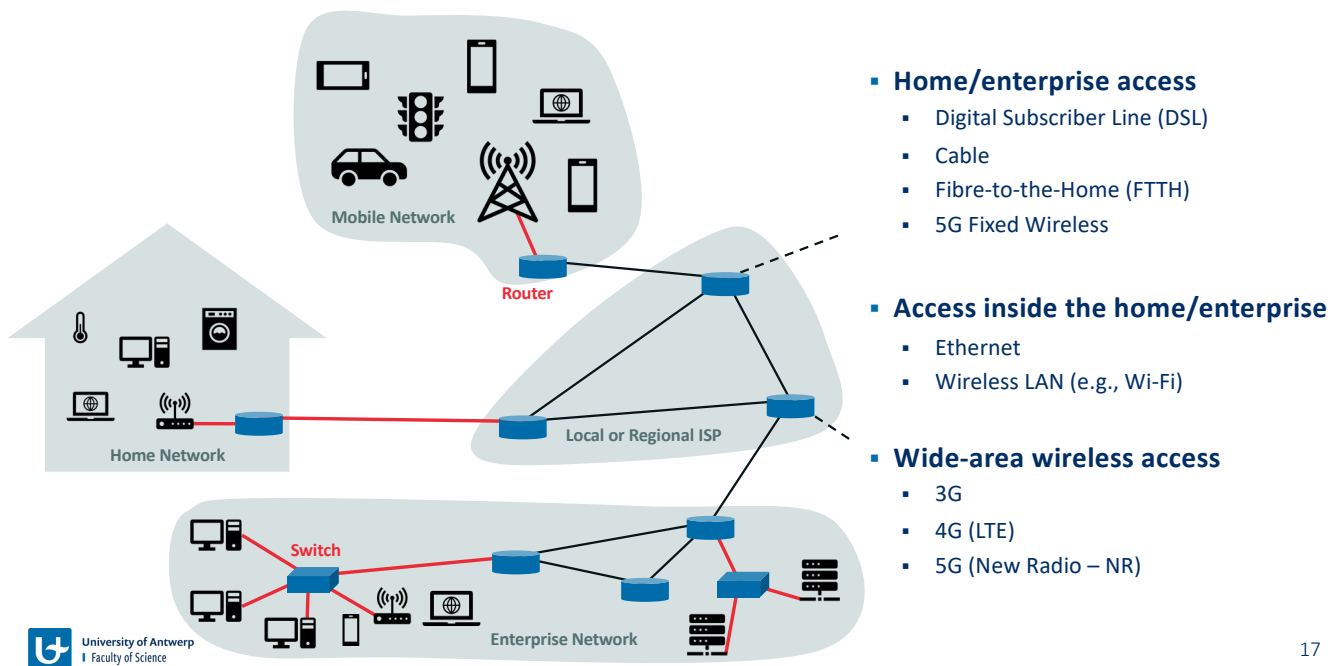
We can also describe the Internet from an entirely different angle—namely, as an infrastructure that provides services to applications. In addition to traditional applications such as email and Web surfing, Internet applications include mobile smartphone and tablet applications, including Internet messaging, mapping with real-time road-traffic information, music streaming movie and television streaming, online social media, video conferencing, multi-person games, and location-based recommendation systems. The applications are said to be distributed applications, since they involve multiple end systems that exchange data with each other. Importantly, Internet applications run on end systems—they do not run in the packet switches in the network core. Although packet switches facilitate the exchange of data among end systems, they are not concerned with the application that is the source or sink of data.

End systems attached to the Internet provide a socket interface that specifies how a program running on one end system asks the Internet infrastructure to deliver data to a specific destination program running on another end system. This Internet socket interface is a set of rules that the sending program must follow so that the Internet can deliver the data to the destination program.

Let's draw upon a simple analogy. Suppose Alice wants to send a letter to Bob using the postal service. Alice, of course, can't just write the letter (the data) and drop the letter out her window. Instead, the postal service requires that Alice put the letter in an envelope; write Bob's full name, address, and zip code in the center of the envelope; seal the envelope; put a stamp in the upper right-hand corner of the envelope; and finally, drop the envelope into an official postal service mailbox. Thus, the postal service

has its own “postal service interface,” or set of rules, that Alice must follow to have the postal service deliver her letter to Bob. In a similar manner, the Internet has a socket interface that the program sending data must follow to have the Internet deliver the data to the program that will receive the data.

Access Networks



17

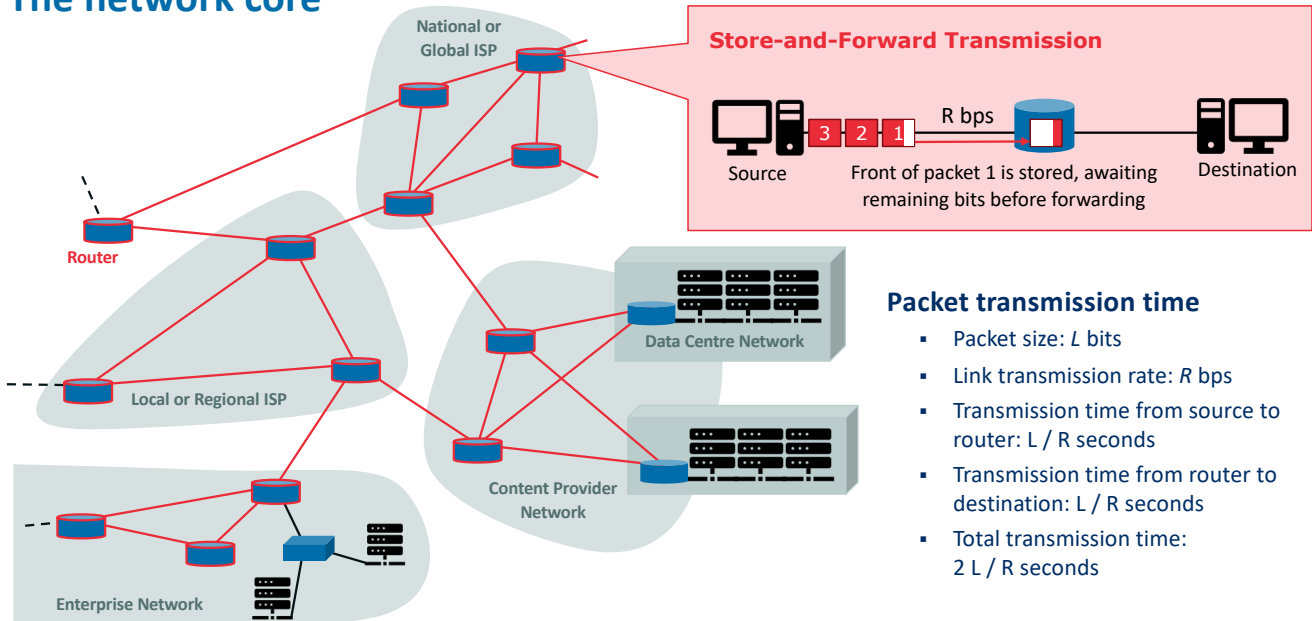
The access network that physically connects an end system to the first router (also known as the “edge router”) on a path from the end system to any other distant end system. The figure shows several types of access networks in thick red lines and the settings (home, enterprise, wide-area mobile wireless) in which they are used.

Today, the two most prevalent types of broadband residential access are **digital subscriber line (DSL)** and cable. A residence typically obtains DSL Internet access from the same local telephone company (telco) that provides its wired local phone access. Thus, when DSL is used, a customer’s telco is also its ISP. While DSL makes use of the telco’s existing local telephone infrastructure, **cable Internet access** makes use of the cable television company’s existing cable television infrastructure. A residence obtains cable Internet access from the same company that provides its cable television. Although DSL and cable networks currently represent the majority of residential broadband access in the United States, an up-and-coming technology that provides even higher speeds is **fibre to the home (FTTH)**. As the name suggests, the FTTH concept is simple—provide an optical fibre path from the central office (CO) directly to the home. FTTH can potentially provide Internet access rates in the gigabits per second range. In addition to DSL, Cable, and FTTH, **5G fixed wireless** is beginning to be deployed. 5G fixed wireless not only promises high-speed residential access but will do so without installing costly and failure-prone cabling from the telco’s CO to the home. With 5G fixed wireless, using beam-forming technology, data is sent wirelessly from a provider’s base station to a modem in the home. A Wi-Fi wireless router is connected to the modem (possibly bundled together), similar to how a Wi-Fi wireless router is connected to a cable or DSL modem.

On corporate and university campuses, and increasingly in home settings, a local area network (LAN) is used to connect an end system to the edge router. Although there are many types of LAN technologies, Ethernet is by far the most prevalent access technology in corporate, university, and home networks. **Ethernet** users use twisted-pair copper wire to connect to an Ethernet switch. The Ethernet switch, or a network of such interconnected switches, is then in turn connected into the larger Internet. A wireless LAN user must typically be within a few tens of meters of the access point. Wireless LAN access based on IEEE 802.11 technology, more colloquially known as Wi-Fi, is now just about everywhere—universities, business offices, cafes, airports, homes, and even in airplanes.

Mobile devices such as iPhones and Android devices are being used to message, share photos in social networks, make mobile payments, watch movies, stream music, and much more while on the run. These devices employ the same wireless infrastructure used for cellular telephony to send/receive packets through a base station that is operated by the cellular network provider. Unlike Wi-Fi, a user need only be within a few tens of kilometres (as opposed to a few tens of meters) of the base station. Telecommunications companies have made enormous investments in so-called fourth-generation (4G) wireless, which provides real-world download speeds of up to 60 Mbps. But even higher-speed wide-area access technologies—a fifth-generation (5G) of wide-area wireless networks—are already being deployed.

The network core



Having examined the Internet's edge, let us now delve more deeply inside the network core—the mesh of packet switches and links that interconnects the Internet's end systems. In a network application, end systems exchange **messages** with each other. Messages can contain anything the application designer wants. Messages may perform a control or can contain data, such as an e-mail message, a JPEG image, or an MP3 audio file. To send a message from a source end system to a destination end system, the source breaks long messages into smaller chunks of data known as **packets**. Between source and destination, each packet travels through communication links and **packet switches** (for which there are two predominant types, **routers** and **link-layer switches**). Packets are transmitted over each communication link at a rate equal to the *full* transmission rate of the link. So, if a source end system or a packet switch is sending a packet of L bits over a link with transmission rate R bits/sec, then the time to transmit the packet is L / R seconds.

Most packet switches use **store-and-forward transmission** at the inputs to the links. Store-and-forward transmission means that the packet switch must receive the entire packet before it can begin to transmit the first bit of the packet onto the outbound link. To explore store-and-forward transmission in more detail, consider a simple network consisting of two end systems connected by a single router. A router will typically have many incident links, since its job is to switch an incoming packet onto an outgoing link; in this simple example, the router has the rather simple task of transferring a packet from one (input) link to the only other attached link. In this example, the source has three packets, each consisting of L bits, to send to the destination. At the snapshot of time shown, the source has transmitted some of packet 1, and the front of packet 1 has

already arrived at the router. Because the router employs store-and-forwarding, at this instant of time, the router cannot transmit the bits it has received; instead it must first buffer (i.e., “store”) the packet’s bits. Only after the router has received *all* of the packet’s bits can it begin to transmit (i.e., “forward”) the packet onto the outbound link.

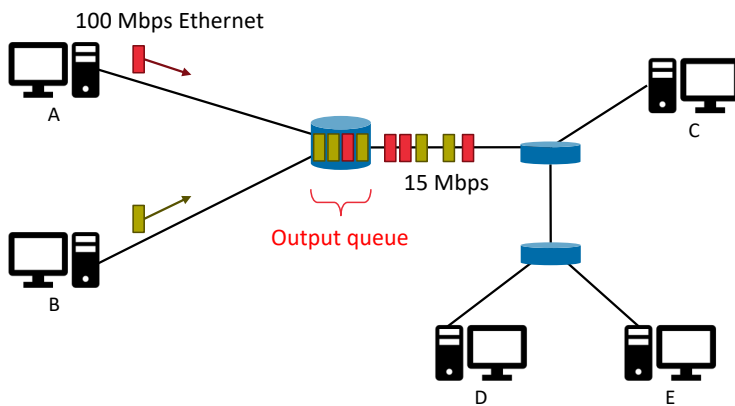
To gain some insight into store-and-forward transmission, let’s now calculate the amount of time that elapses from when the source begins to send the packet until the destination has received the entire packet. Here we will ignore propagation delay—the time it takes for the bits to travel across the wire at near the speed of light. The source begins to transmit at time 0; at time L/R seconds, the source has transmitted the entire packet, and the entire packet has been received and stored at the router (since there is no propagation delay). At time L/R seconds, since the router has just received the entire packet, it can begin to transmit the packet onto the outbound link towards the destination; at time $2L/R$, the router has transmitted the entire packet, and the entire packet has been received by the destination. Thus, the total delay is $2L/R$. If the switch instead forwarded bits as soon as they arrive (without first receiving the entire packet), then the total delay would be L/R since bits are not held up at the router. However, routers need to receive, store, and *process* the entire packet before forwarding.

Now let’s calculate the amount of time that elapses from when the source begins to send the first packet until the destination has received all three packets. As before, at time L/R , the router begins to forward the first packet. But also at time L/R the source will begin to send the second packet, since it has just finished sending the entire first packet. Thus, at time $2L/R$, the destination has received the first packet and the router has received the second packet. Similarly, at time $3L/R$, the destination has received the first two packets and the router has received the third packet. Finally, at time $4L/R$ the destination has received all three packets!

Let’s now consider the general case of sending one packet from source to destination over a path consisting of N links each of rate R (thus, there are $N-1$ routers between source and destination). Applying the same logic as above, we see that the end-to-end delay is:

$$d_{end-to-end} = N \frac{L}{R}$$

Packet switching



Output buffer (or queue)

- Temporarily stores packets while associated link is busy
- Causes **queuing delays**
- If a packet arrives to a full buffer, **packet loss** occurs

Forwarding tables & routing protocols

- Determine how packets are forwarded by routers
- Each host is identified by an IP address
- **Forwarding tables** map destination IP to output links
- **Routing protocols** configure the forwarding tables

Each packet switch has multiple links attached to it. For each attached link, the packet switch has an **output buffer** (also called an **output queue**), which stores packets that the router is about to send into that link. The output buffers play a key role in packet switching. If an arriving packet needs to be transmitted onto a link but finds the link busy with the transmission of another packet, the arriving packet must wait in the output buffer. Thus, in addition to the store-and-forward delays, packets suffer output buffer **queuing delays**. These delays are variable and depend on the level of congestion in the network. Since the amount of buffer space is finite, an arriving packet may find that the buffer is completely full with other packets waiting for transmission. In this case, **packet loss** will occur—either the arriving packet or one of the already-queued packets will be dropped.

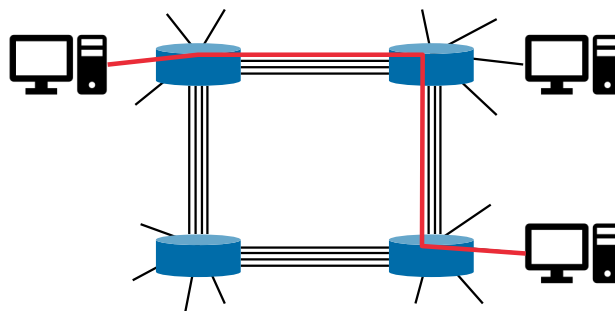
Suppose Hosts A and B are sending packets to Host E. Hosts A and B first send their packets along 100 Mbps Ethernet links to the first router. The router then directs these packets to the 15 Mbps link. If, during a short interval of time, the arrival rate of packets to the router (when converted to bits per second) exceeds 15 Mbps, congestion will occur at the router as packets queue in the link's output buffer before being transmitted onto the link. For example, if Host A and B each send a burst of five packets back-to-back at the same time, then most of these packets will spend some time waiting in the queue. The situation is, in fact, entirely analogous to many common-day situations—for example, when we wait in line for a bank teller or wait in front of a tollbooth.

How does the router determine which link it should forward the packet onto? Packet forwarding is actually done in different ways in different types of computer networks.

Here, we briefly describe how it is done in the Internet. In the Internet, every end system has an address called an IP address. When a source end system wants to send a packet to a destination end system, the source includes the destination's IP address in the packet's header. As with postal addresses, this address has a hierarchical structure. When a packet arrives at a router in the network, the router examines a portion of the packet's destination address and forwards the packet to an adjacent router. More specifically, each router has a **forwarding table** that maps destination addresses (or portions of the destination addresses) to that router's outbound links. When a packet arrives at a router, the router examines the address and searches its forwarding table, using this destination address, to find the appropriate outbound link. The router then directs the packet to this outbound link. the Internet has a number of special **routing protocols** that are used to automatically set the forwarding tables. A routing protocol may, for example, determine the shortest path from each router to each destination and use the shortest path results to configure the forwarding tables in the routers.

Circuit switching

- Alternative to packet switching
- Resources needed along the path (buffers, link transmission rate) to provide communication between the end systems are **reserved** for the duration of the communication session
- Example: Traditional telephone networks



There are two fundamental approaches to moving data through a network of links and switches: **circuit switching** and **packet switching**. Having covered packet-switched networks in the previous subsection, we now turn our attention to circuit-switched networks.

In circuit-switched networks, the resources needed along a path (buffers, link transmission rate) to provide for communication between the end systems are *reserved* for the duration of the communication session between the end systems. In packet-switched networks, these resources are *not* reserved; a session's messages use the resources on demand and, as a consequence, may have to wait (that is, queue) for access to a communication link. As a simple analogy, consider two restaurants, one that requires reservations and another that neither requires reservations nor accepts them. For the restaurant that requires reservations, we have to go through the hassle of calling before we leave home. But when we arrive at the restaurant we can, in principle, immediately be seated and order our meal. For the restaurant that does not require reservations, we don't need to bother to reserve a table. But when we arrive at the restaurant, we may have to wait for a table before we can be seated.

Traditional telephone networks are examples of circuit-switched networks. Consider what happens when one person wants to send information (voice or facsimile) to another over a telephone network. Before the sender can send the information, the network must establish a connection between the sender and the receiver. This is a *bona fide* connection for which the switches on the path between the sender and receiver maintain connection state for that connection. In the jargon of telephony, this

connection is called a **circuit**. When the network establishes the circuit, it also reserves a constant transmission rate in the network's links (representing a fraction of each link's transmission capacity) for the duration of the connection. Since a given transmission rate has been reserved for this sender-to-receiver connection, the sender can transfer the data to the receiver at the *guaranteed* constant rate.

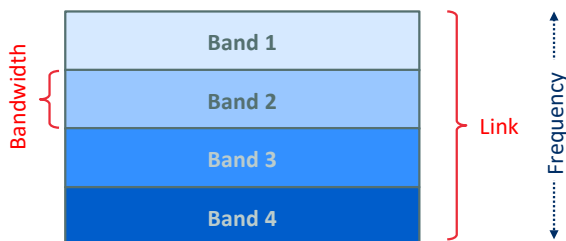
The figure illustrates a circuit-switched network. In this network, the four circuit switches are interconnected by four links. Each of these links has four circuits, so that each link can support four simultaneous connections. The hosts (for example, PCs and workstations) are each directly connected to one of the switches. When two hosts want to communicate, the network establishes a dedicated **end-to-end connection** between the two hosts. Thus, in order for Host A to communicate with Host B, the network must first reserve one circuit on each of two links. In this example, the dedicated end-to-end connection uses the second circuit in the first link and the fourth circuit in the second link. Because each link has four circuits, for each link used by the end-to-end connection, the connection gets one fourth of the link's total transmission capacity for the duration of the connection.

Thus, for example, if each link between adjacent switches has a transmission rate of 1 Mbps, then each end-to-end circuit-switch connection gets 250 kbps of dedicated transmission rate.

In contrast, consider what happens when one host wants to send a packet to another host over a packet-switched network, such as the Internet. As with circuit switching, the packet is transmitted over a series of communication links. But different from circuit switching, the packet is sent into the network without reserving any link resources whatsoever. If one of the links is congested because other packets need to be transmitted over the link at the same time, then the packet will have to wait in a buffer at the sending side of the transmission link and suffer a delay. The Internet makes its best effort to deliver packets in a timely manner, but it does not make any guarantees.

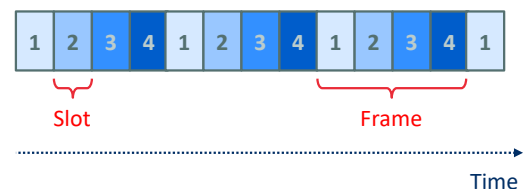
Multiplexing multiple circuits over one link

Frequency Division Multiplexing (FDM)



Each frequency band is dedicated to a specific sender-receiver pair

Time Division Multiplexing (TDM)



All slots with the same label are dedicated to a specific sender-receiver pair

A circuit in a link is implemented with either **frequency-division multiplexing (FDM)** or **time-division multiplexing (TDM)**. With FDM, the frequency spectrum of a link is divided up among the connections established across the link. Specifically, the link dedicates a frequency band to each connection for the duration of the connection. In telephone networks, this frequency band typically has a width of 4 kHz (that is, 4,000 hertz or 4,000 cycles per second). The width of the band is called, not surprisingly, the **bandwidth**. FM radio stations also use FDM to share the frequency spectrum between 88 MHz and 108 MHz, with each station being allocated a specific frequency band.

For a TDM link, time is divided into frames of fixed duration, and each frame is divided into a fixed number of time slots. When the network establishes a connection across a link, the network dedicates one time slot in every frame to this connection. These slots are dedicated for the sole use of that connection, with one time slot available for use (in every frame) to transmit the connection's data.

The figures illustrate FDM and TDM for a specific network link supporting up to four circuits. For FDM, the frequency domain is segmented into four bands, each of a fixed equal bandwidth. For TDM, the time domain is segmented into frames, with four time slots in each frame; each circuit is assigned the same dedicated slot in the revolving TDM frames. For TDM, the transmission rate of a circuit is equal to the frame rate multiplied by the number of bits in a slot. For example, if the link transmits 8,000 frames per second and each slot consists of 8 bits, then the transmission rate of each circuit is 64 kbps.

Is packet switching or circuit switching better?

Packet switching is better, as it offers better capacity sharing

Packet switching is better, as it is simpler and cheaper to implement

Circuit switching is better, as it provides more predictable end-to-end delays for real-time services

They both have their advantages and disadvantages

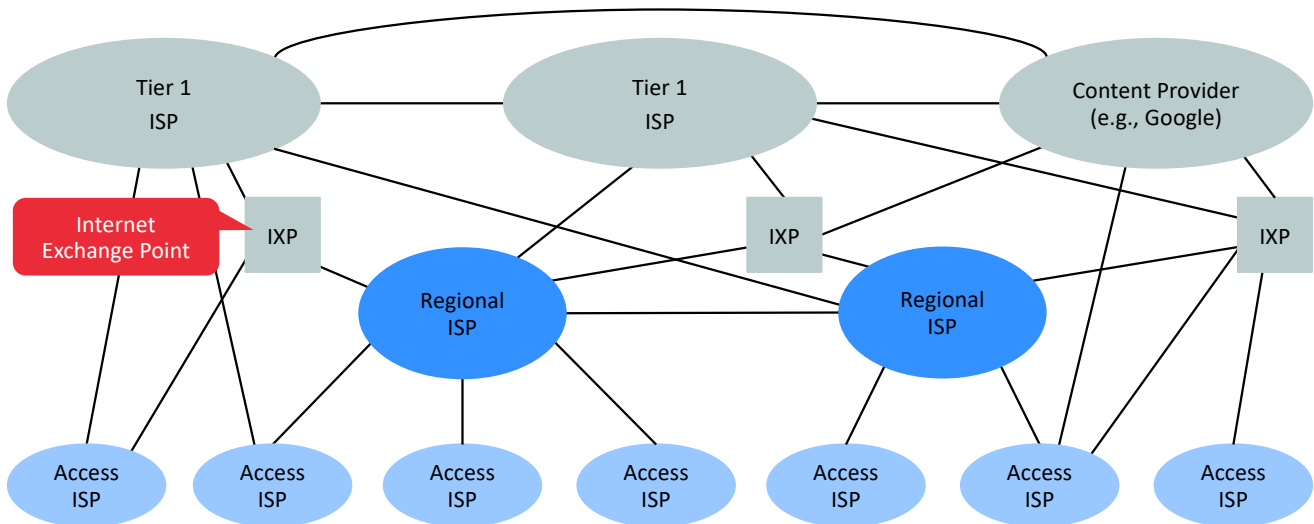
Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Is packet switching or circuit switching better?

https://www.polleverywhere.com/multiple_choice_polls/EkgkHd0JyiqQcd0lfLsK7?display_state=chart&activity_state=opened&state=opened&flow=Default&onscreen=persist

The Internet: A network of networks



Tier 1 Internet Service Providers (ISPs) are **global transit ISPs**. They offer Internet connectivity in many places across the globe. The global transit ISPs themselves must interconnect, however: Otherwise access ISPs connected to one of the global transit providers would not be able to communicate with access ISPs connected to the other global transit providers. There are approximately a dozen tier-1 ISPs, including Level 3 Communications, AT&T, Sprint, and NTT. In reality, although some ISPs do have impressive global coverage and do directly connect with many access ISPs, no ISP has presence in each and every city in the world. Instead, in any given region, there may be a **regional ISP** to which the access ISPs in the region connect. Each regional ISP then connects to **tier-1 ISPs**. Each access ISP pays the regional ISP to which it connects, and each regional ISP pays the tier-1 ISP to which it connects. (An access ISP can also connect directly to a tier-1 ISP, in which case it pays the tier-1 ISP). Thus, there is customer-provider relationship at each level of the hierarchy. Note that the tier-1 ISPs do not pay anyone as they are at the top of the hierarchy. To further complicate matters, in some regions, there may be a larger regional ISP (possibly spanning an entire country) to which the smaller regional ISPs in that region connect; the larger regional ISP then connects to a tier-1 ISP.

Additionally, the Internet contains points of presence (PoPs), multi-homing, peering, and Internet exchange points (IXPs). PoPs exist in all levels of the hierarchy, except for the bottom (access ISP) level. A **PoP** is simply a group of one or more routers (at the same location) in the provider's network where customer ISPs can connect into the provider ISP. For a customer network to connect to a provider's PoP, it can lease a high-speed link from a third-party telecommunications provider to directly connect one of its routers to

a router at the PoP. Any ISP (except for tier-1 ISPs) may choose to **multi-home**, that is, to connect to two or more provider ISPs. So, for example, an access ISP may multi-home with two regional ISPs, or it may multi-home with two regional ISPs and also with a tier-1 ISP. Similarly, a regional ISP may multi-home with multiple tier-1 ISPs. When an ISP multi-homes, it can continue to send and receive packets into the Internet even if one of its providers has a failure.

The amount that a customer ISP pays a provider ISP reflects the amount of traffic it exchanges with the provider. To reduce these costs, a pair of nearby ISPs at the same level of the hierarchy can **peer**, that is, they can directly connect their networks together so that all the traffic between them passes over the direct connection rather than through upstream intermediaries. When two ISPs peer, it is typically settlement-free, that is, neither ISP pays the other. As noted earlier, tier-1 ISPs also peer with one another, settlement-free. Along these same lines, a third-party company can create an **Internet Exchange Point (IXP)**, which is a meeting point where multiple ISPs can peer together. An IXP is typically in a stand-alone building with its own switches. There are over 600 IXPs in the Internet today.

Finally, the Internet contains **content-provider networks**. Google is currently one of the leading examples of such a content-provider network. Currently, Google has 19 major data centers distributed across North America, Europe, Asia, South America, and Australia with each data center having tens or hundreds of thousands of servers. Additionally, Google has smaller data centers, each with a few hundred servers; these smaller data centers are often located within IXPs. The Google data centers are all interconnected via Google's private TCP/IP network, which spans the entire globe but is nevertheless separate from the public Internet. Importantly, the Google private network only carries traffic to/from Google servers. As shown in the figure, the Google private network attempts to "bypass" the upper tiers of the Internet by peering (settlement-free) with lower-tier ISPs, either by directly connecting with them or by connecting with them at IXPs. However, because many access ISPs can still only be reached by transiting through tier-1 networks, the Google network also connects to tier-1 ISPs, and pays those ISPs for the traffic it exchanges with them. By creating its own network, a content provider not only reduces its payments to upper-tier ISPs, but also has greater control of how its services are ultimately delivered to end users.

In summary, today's Internet—a network of networks—is complex, consisting of a dozen or so tier-1 ISPs and hundreds of thousands of lower-tier ISPs. The ISPs are diverse in their coverage, with some spanning multiple continents and oceans, and others limited to narrow geographic regions. The lower-tier ISPs connect to the higher-tier ISPs, and the higher-tier ISPs interconnect with one another. Users and content providers are customers of lower-tier ISPs, and lower-tier ISPs are customers of higher-tier ISPs. In recent years, major content providers have also created their own networks and connect directly into lower-tier ISPs where possible.

Network Performance

Given a packet size of L bits, and transmission rate of R bps, how long would it take to transmit 3 packets from source to destination with 1 intermediary router?

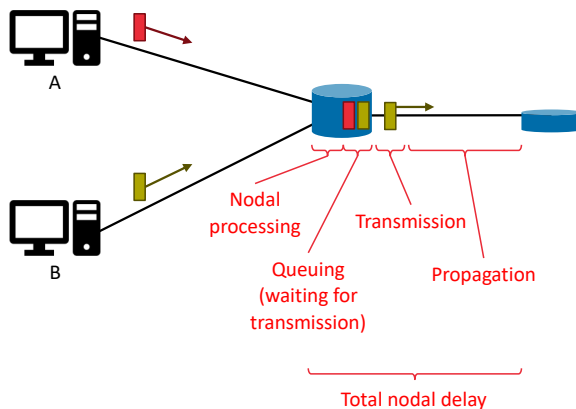
- 2 L / R seconds
- 3 L / R seconds
- 4 L / R seconds
- 5 L / R seconds
- 6 L / R seconds

Start the presentation to see live content. For screen share software, share the entire screen. Get help at pollev.com/app

Poll Title: Do not modify the notes in this section to avoid tampering with the Poll Everywhere activity.
More info at polleverywhere.com/support

Given a packet size of L bits, and transmission rate of R bps, how long would it take to transmit 3 packets from source to destination with 1 intermediary router?
https://www.polleverywhere.com/multiple_choice_polls/aTZjYTd740SeXfuqWUrrj?display_state=chart&activity_state=opened&state=opened&flow=Default&onscreen=persist

Types of nodal delay



- **Processing delay**

- The time required to examine the packet's header and determine where to direct the packet

- **Queuing delay**

- The time the packet spends waiting in the buffer to be transmitted
- Depends on the number of earlier-arriving packets waiting

- **Transmission delay**

- The time to transmit the packet on the link
- Depends on the packet size L and transmission rate R

- **Propagation delay**

- The time required for bits to propagate from the source to destination router of the link (limited by speed of light)
- Propagation speed s is usually between 2 and 3×10^8 m/s
- Propagation delay equals d/s , with d the link distance

Recall that a packet starts in a host (the source), passes through a series of routers, and ends its journey in another host (the destination). As a packet travels from one node (host or router) to the subsequent node (host or router) along this path, the packet suffers from several types of delays at *each* node along the path. The most important of these delays are the **nodal processing delay**, **queuing delay**, **transmission delay**, and **propagation delay**; together, these delays accumulate to give a **total nodal delay**. The performance of many Internet applications—such as search, Web browsing, e-mail, maps, instant messaging, and voice-over-IP—are greatly affected by network delays. In order to acquire a deep understanding of packet switching and computer networks, we must understand the nature and importance of these delays.

Processing Delay

The time required to examine the packet's header and determine where to direct the packet is part of the **processing delay**. The processing delay can also include other factors, such as the time needed to check for bit-level errors in the packet that occurred in transmitting the packet's bits from the upstream node to router A. Processing delays in high-speed routers are typically on the order of microseconds or less. After this nodal processing, the router directs the packet to the queue that precedes the link to router B.

Queuing Delay

At the queue, the packet experiences a **queuing delay** as it waits to be transmitted onto the link. The length of the queuing delay of a specific packet will depend on the number of earlier-arriving packets that are queued and waiting for transmission onto the link. If the queue is empty and no other packet is currently being transmitted, then our packet's

queuing delay will be zero. On the other hand, if the traffic is heavy and many other packets are also waiting to be transmitted, the queuing delay will be long. We will see shortly that the number of packets that an arriving packet might expect to find is a function of the intensity and nature of the traffic arriving at the queue. Queuing delays can be on the order of microseconds to milliseconds in practice.

Transmission Delay

Assuming that packets are transmitted in a first-come-first-served manner, as is common in packet-switched networks, our packet can be transmitted only after all the packets that have arrived before it have been transmitted. Denote the length of the packet by L bits, and denote the transmission rate of the link from router A to router B by R bits/sec. For example, for a 10 Mbps Ethernet link, the rate is $R = 10$ Mbps; for a 100 Mbps Ethernet link, the rate is $R = 100$ Mbps. The **transmission delay** is L/R . This is the amount of time required to push (that is, transmit) all of the packet's bits into the link. Transmission delays are typically on the order of microseconds to milliseconds in practice.

Propagation Delay

Once a bit is pushed into the link, it needs to propagate to router B. The time required to propagate from the beginning of the link to router B is the **propagation delay**. The bit propagates at the propagation speed of the link. The propagation speed depends on the physical medium of the link (that is, fiber optics, twisted-pair copper wire, and so on) and is in the range of 2×10^8 meters/sec to 3×10^8 meters/sec, which is equal to, or a little less than, the speed of light. The propagation delay is the distance between two routers divided by the propagation speed. That is, the propagation delay is d/s , where d is the distance between router A and router B and s is the propagation speed of the link. Once the last bit of the packet propagates to node B, it and all the preceding bits of the packet are stored in router B. The whole process then continues with router B now performing the forwarding. In wide-area networks, propagation delays are on the order of milliseconds.

End-to-end delay

- Packets accumulate nodal delay from all routers they pass on the path from source to destination
- Assuming a path with $N - 1$ routers between source and destination

$$d_{e2e} = \sum_{n=1}^N d_{n,proc} + d_{n,queue} + d_{n,trans} + d_{n,prop}$$

- End-to-end delay can be analysed using the Traceroute programme
 - Sends 3 packets to each intermediary router, and determines delay to that router based on the response
 - Standardized as IETF RFC 1393

```
> traceroute www.google.be
Tracing route to www.google.be [2a00:1450:400e:801::2003]
over a maximum of 30 hops:
  0  0 ms    0 ms    0 ms    ptr-g10bpkxncsi2klp2fmu.18120a2.ip6.access.telenet.be [2a02:1812:162d:6a00:5667:51ff:fedd:a8a6]
  1  27 ms   15 ms   13 ms   ptr-377whsjxcbe4fbcig8ap.18120a2.ip6.access.telenet.be [2a02:181f:0:80a3::1]
  2  *      *      *      Request timed out.
  ...
 13 37 ms    *      *      2001:4860:0:1::4fbd
 14 30 ms    40 ms   32 ms   2001:4860:0:1::4fc7
 15 60 ms    35 ms   34 ms   ams17s10-in-x03.1e100.net [2a00:1450:400e:801::2003]
Trace complete.
```

Our discussion up to this point has focused on the nodal delay, that is, the delay at a single router. Let's now consider the total delay from source to destination. To get a handle on this concept, suppose there are $N - 1$ routers between the source host and the destination host. Let's also suppose for the moment that the network is uncongested (so that queuing delays are negligible), the processing delay at each router and at the source host is d_{proc} , the transmission rate out of each router and out of the source host is R bits/sec, and the propagation on each link is d_{prop} . The nodal delays accumulate and give an end-to-end delay,

$$d_{e2e} = \sum_{n=1}^N d_{n,proc} + d_{n,queue} + d_{n,trans} + d_{n,prop}$$

Where, once again, $d_{trans} = L/R$, where L is the packet size.

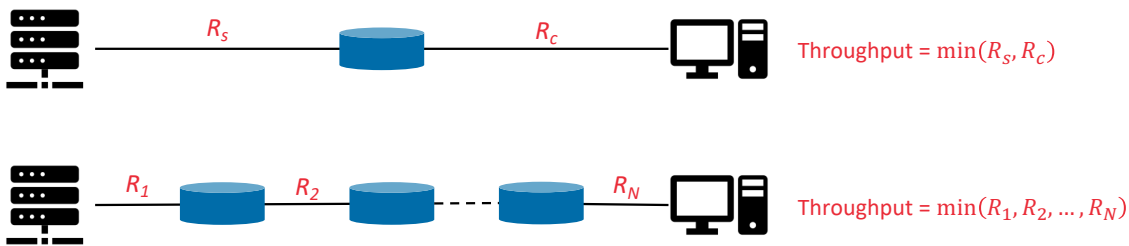
To get a hands-on feel for end-to-end delay in a computer network, we can make use of the **Traceroute program**. Traceroute is a simple program that can run in any Internet host. When the user specifies a destination hostname, the program in the source host sends multiple, special packets toward that destination. As these packets work their way toward the destination, they pass through a series of routers. When a router receives one of these special packets, it sends back to the source a short message that contains the name and address of the router. More specifically, suppose there are $N - 1$ routers between the source and the destination. Then the source will send N special packets into the network, with each packet addressed to the ultimate destination. These N special packets are marked 1 through N , with the first packet marked 1 and the last packet marked N . When the n^{th} router receives the n^{th} packet marked n , the router does

not forward the packet toward its destination, but instead sends a message back to the source. When the destination host receives the N^{th} packet, it too returns a message back to the source. The source records the time that elapses between when it sends a packet and when it receives the corresponding return message; it also records the name and address of the router (or the destination host) that returns the message. In this manner, the source can reconstruct the route taken by packets flowing from source to destination, and the source can determine the round-trip delays to all the intervening routers.

Traceroute actually repeats the experiment just described three times, so the source actually sends $3 \cdot N$ packets to the destination. Because the queuing delay is varying with time, the round-trip delay of packet n sent to a router n can sometimes be longer than the round-trip delay of packet $n+1$ sent to router $n+1$. RFC 1393 describes Traceroute in detail.

Throughput

- **Instantaneous throughput:** The rate (in bits/sec) at which a host is receiving data at a specific time
- **Average throughput:** The average rate (in bits/sec) at which an entire file is downloaded
 - If a file consists of F bits and the transfer takes T seconds, the average throughput is F/T bits/sec
- **Bottleneck link:** The link on the path from source to destination with the lowest transmission rate



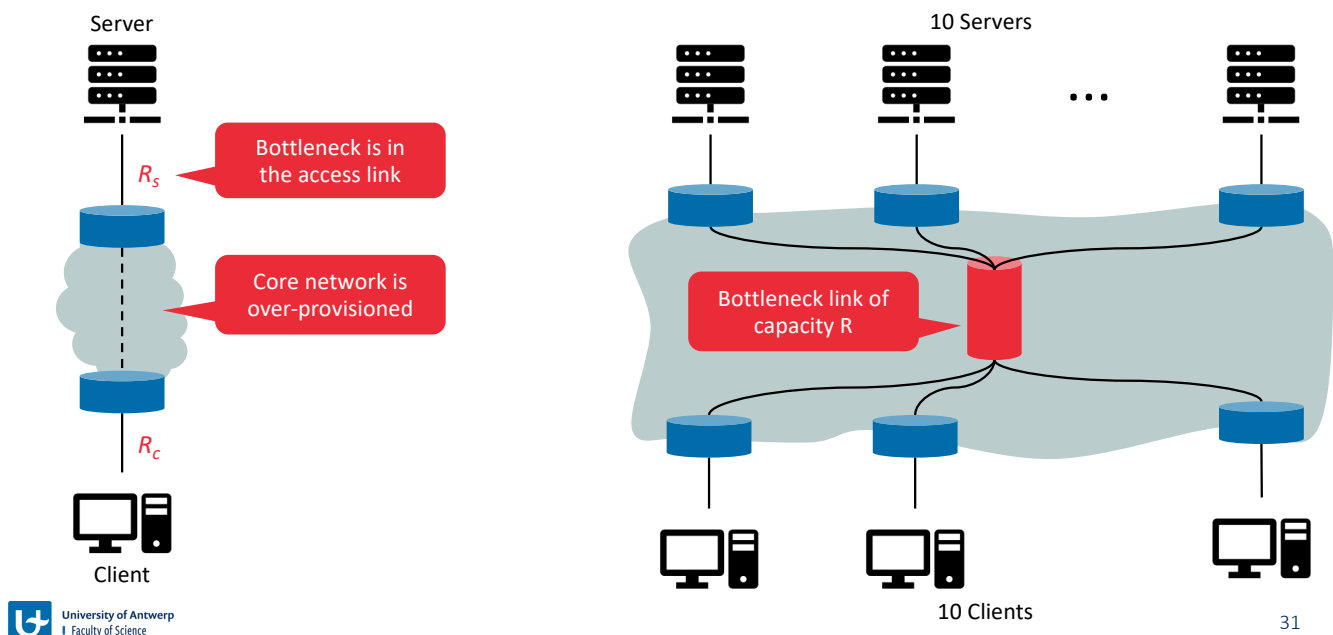
In addition to delay and packet loss, another critical performance measure in computer networks is end-to-end throughput. To define throughput, consider transferring a large file from Host A to Host B across a computer network. This transfer might be, for example, a large video clip from one computer to another. The **instantaneous throughput** at any instant of time is the rate (in bits/sec) at which Host B is receiving the file. (Many applications display the instantaneous throughput during downloads in the user interface—perhaps you have observed this before! You might like to try measuring the end-to-end delay and download throughput between your and servers around the Internet using the speedtest application) If the file consists of F bits and the transfer takes T seconds for Host B to receive all F bits, then the **average throughput** of the file transfer is F/T bits/sec. For some applications, such as Internet telephony, it is desirable to have a low delay and an instantaneous throughput consistently above some threshold (for example, over 24 kbps for some Internet telephony applications and over 256 kbps for some real-time video applications). For other applications, including those involving file transfers, delay is not critical, but it is desirable to have the highest possible throughput.

To gain further insight into the important concept of throughput, let's consider a few examples. The top figure shows two end systems, a server and a client, connected by two communication links and a router. Consider the throughput for a file transfer from the server to the client. Let R_s denote the rate of the link between the server and the router; and R_c denote the rate of the link between the router and the client. Suppose that the only bits being sent in the entire network are those from the server to the client. We now ask, in this ideal scenario, what is the server-to-client throughput? To

answer this question, we may think of bits as *fluid* and communication links as *pipes*. Clearly, the server cannot pump bits through its link at a rate faster than R_s bps; and the router cannot forward bits at a rate faster than R_c bps. If $R_s < R_c$, then the bits pumped by the server will “flow” right through the router and arrive at the client at a rate of R_s bps, giving a throughput of R_s bps. If, on the other hand, $R_c < R_s$, then the router will not be able to forward bits as quickly as it receives them. In this case, bits will only leave the router at rate R_c , giving an end-to-end throughput of R_c . (Note also that if bits continue to arrive at the router at rate R_s , and continue to leave the router at R_c , the backlog of bits at the router waiting for transmission to the client will grow and grow—a most undesirable situation!) Thus, for this simple two-link network, the throughput is $\min\{R_c, R_s\}$, that is, it is the transmission rate of the **bottleneck link**.

The bottom figure now shows a network with N links between the server and the client, with the transmission rates of the N links being $R_1, R_2, \dots, R_N$. Applying the same analysis as for the two-link network, we find that the throughput for a file transfer from server to client is $\min\{R_1, R_2, \dots, R_N\}$, which is once again the transmission rate of the bottleneck link along the path between server and client.

Throughput depends on transmission rate and co-existing traffic



Now consider another example motivated by today's Internet. The left figure shows two end systems, a server and a client, connected to a computer network. Consider the throughput for a file transfer from the server to the client. The server is connected to the network with an access link of rate R_s and the client is connected to the network with an access link of rate R_c . Now suppose that all the links in the core of the communication network have very high transmission rates, much higher than R_s and R_c . Indeed, today, the core of the Internet is over-provisioned with high speed links that experience little congestion. Also suppose that the only bits being sent in the entire network are those from the server to the client. Because the core of the computer network is like a wide pipe in this example, the rate at which bits can flow from source to destination is again the minimum of R_s and R_c , that is, throughput = $\min\{R_s, R_c\}$. Therefore, the constraining factor for throughput in today's Internet is typically the access network.

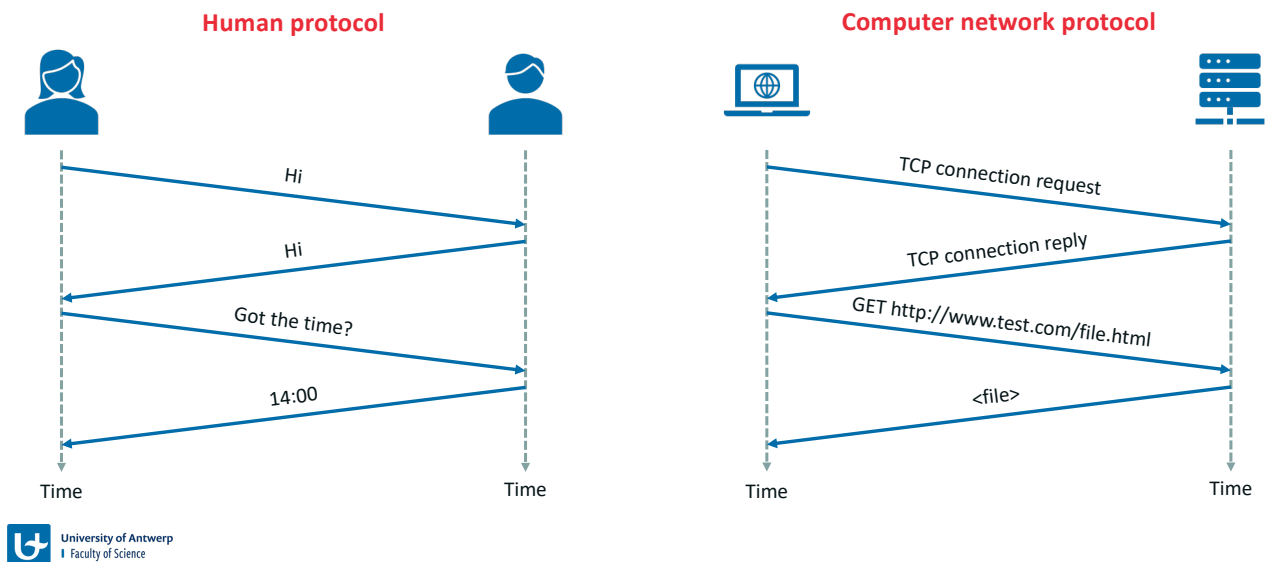
For a final example, consider the right figure in which there are 10 servers and 10 clients connected to the core of the computer network. In this example, there are 10 simultaneous downloads taking place, involving 10 client-server pairs. Suppose that these 10 downloads are the only traffic in the network at the current time. As shown in the figure, there is a link in the core that is traversed by all 10 downloads. Denote R for the transmission rate of this link R . Let's suppose that all server access links have the same rate R_s , all client access links have the same rate R_c , and the transmission rates of all the links in the core—except the one common link of rate R —are much larger than R_s , R_c , and R . Now we ask, what are the throughputs of the downloads? Clearly, if the rate of the common link, R , is large—say a hundred times larger than both R_s and R_c —then

the throughput for each download will once again be $\min\{R_s, R_c\}$. But what if the rate of the common link is of the same order as R_s and R_c ? What will the throughput be in this case? Let's take a look at a specific example. Suppose $R_s = 2$ Mbps, $R_c = 1$ Mbps, $R = 5$ Mbps, and the common link divides its transmission rate equally among the 10 downloads. Then the bottleneck for each download is no longer in the access network, but is now instead the shared link in the core, which only provides each download with 500 kbps of throughput. Thus, the end-to-end throughput for each download is now reduced to 500 kbps.

The examples show that throughput depends on the transmission rates of the links over which the data flows. We saw that when there is no other intervening traffic, the throughput can simply be approximated as the minimum transmission rate along the path between source and destination. The example in the right figure shows that more generally the throughput depends not only on the transmission rates of the links along the path, but also on the intervening traffic. In particular, a link with a high transmission rate may nonetheless be the bottleneck link for a file transfer if many other data flows are also passing through that link.

Protocols

Protocols define the conventions of communication between entities



Consider what you do when you want to ask someone for the time of day. Human protocol (or good manners, at least) dictates that one first offer a greeting (the first “Hi”) to initiate communication with someone else. The typical response to a “Hi” is a returned “Hi” message. Implicitly, one then takes a cordial “Hi” response as an indication that one can proceed and ask for the time of day. A different response to the initial “Hi” (such as “Don’t bother me!” or “I don’t speak English,” or some unprintable reply) might indicate an unwillingness or inability to communicate. In this case, the human protocol would be not to ask for the time of day. Sometimes one gets no response at all to a question, in which case one typically gives up asking that person for the time. Note that in our human protocol, *there are specific messages we send, and specific actions we take in response to the received reply messages or other events* (such as no reply within some given amount of time). Clearly, transmitted and received messages, and actions taken when these messages are sent or received or other events occur, play a central role in a human protocol. If people run different protocols (for example, if one person has manners but the other does not, or if one understands the concept of time and the other does not) the protocols do not interoperate and no useful work can be accomplished.

The same is true in networking—it takes two (or more) communicating entities running the same protocol in order to accomplish a task. A network protocol is similar to a human protocol, except that the entities exchanging messages and taking actions are hardware or software components of some device (for example, computer, smartphone, tablet, router, or other network-capable device). All activity in the Internet that involves two or more communicating remote entities is governed by a protocol. For example,

hardware-implemented protocols in two physically connected computers control the flow of bits on the “wire” between the two network interface cards; congestion-control protocols in end systems control the rate at which packets are transmitted between sender and receiver; protocols in routers determine a packet’s path from source to destination. Protocols are running everywhere in the Internet.

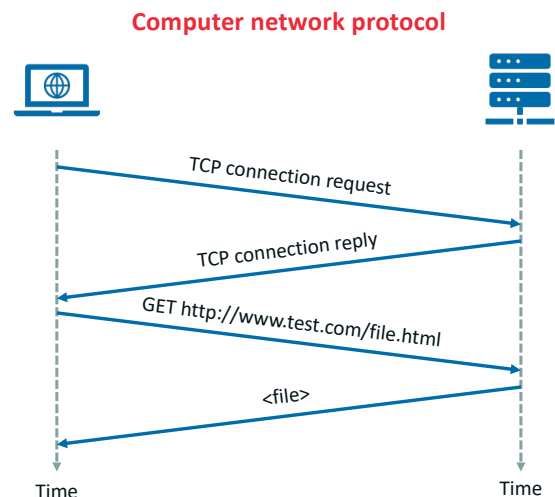
As an example of a computer network protocol with which you are probably familiar, consider what happens when you make a request to a Web server, that is, when you type the URL of a Web page into your Web browser. The scenario is illustrated in the right half of the figure. First, your computer will send a connection request message to the Web server and wait for a reply. The Web server will eventually receive your connection request message and return a connection reply message. Knowing that it is now OK to request the Web document, your computer then sends the name of the Web page it wants to fetch from that Web server in a GET message. Finally, the Web server returns the Web page (file) to your computer.

Definition of a computer network protocol

- “A protocol defines the **format and the order of messages exchanged** between two or more communicating entities, as well as the **actions taken** on the transmission and/or receipt of a message or other event.”

- Standardization of network protocols

- Internet Engineering Task Force (IETF)
 - Standards documents called Requests for Comments (RFCs)
 - TCP, IP, HTTP, TLS, IPSec, UDP, BGP, OSPF, ...
- Institute of Electrical and Electronics Engineers (IEEE)
 - IEEE 802.11 (Wi-Fi), IEEE 802.3 (Ethernet), ...
- 3rd Generation Partnership Project (3GPP)
 - 1G, 2G, 3G, 4G (LTE), 5G, ...
- ETSI
 - Network management, network virtualization, ...



End systems, packet switches, and other pieces of the Internet run **protocols** that control the sending and receiving of information within the Internet. The **Transmission Control Protocol (TCP)** and the **Internet Protocol (IP)** are two of the most important protocols in the Internet. The IP protocol specifies the format of the packets that are sent and received among routers and end systems. The Internet's principal protocols are collectively known as **TCP/IP**. Given the importance of protocols to the Internet, it's important that everyone agree on what each and every protocol does, so that people can create systems and products that interoperate. This is where standards come into play. **Internet standards** are developed by the Internet Engineering Task Force (IETF). The IETF standards documents are called **requests for comments (RFCs)**. RFCs started out as general requests for comments (hence the name) to resolve network and protocol design problems that faced the precursor to the Internet.

The Internet protocol stack consists of 5 layers

Application
Transport
Network
Link
Physical

Application Layer

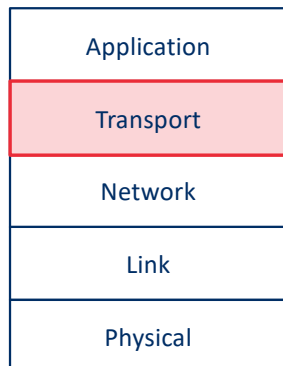
- Distributed over multiple end-systems
- Consists of the applications and their associated application-layer protocols
- Examples
 - HyperText Transfer Protocol (HTTP)
 - Simple Mail Transfer Protocol (SMTP)
 - File Transfer Protocol (FTP)
 - Domain Name System (DNS)
 - ...
- Packet of information is referred to as a **message** at this layer

To provide structure to the design of network protocols, network designers organize protocols—and the network hardware and software that implement the protocols—in **layers**. Each protocol belongs to one of the layers. We are interested in the **services** that a layer offers to the layer above—the so-called **service model** of a layer. Each layer provides its service by (1) performing certain actions within that layer and by (2) using the services of the layer directly below it. For example, the services provided by layer n may include reliable delivery of messages from one edge of the network to the other. This might be implemented by using an unreliable edge-to-edge message delivery service of layer $n - 1$, and adding layer n functionality to detect and retransmit lost messages. A protocol layer can be implemented in software, in hardware, or in a combination of the two. Application-layer protocols—such as HTTP and SMTP—are almost always implemented in software in the end systems; so are transport-layer protocols. Because the physical layer and data link layers are responsible for handling communication over a specific link, they are typically implemented in a network interface card (for example, Ethernet or Wi-Fi interface cards) associated with a given link. The network layer is often a mixed implementation of hardware and software. When taken together, the protocols of the various layers are called the **protocol stack**. The Internet protocol stack consists of five layers: the physical, link, network, transport, and application layers.

The application layer is where network applications and their application-layer protocols reside. The Internet's application layer includes many protocols, such as the HTTP protocol (which provides for Web document request and transfer), SMTP (which provides for the transfer of e-mail messages), and FTP (which provides for the transfer of

files between two end systems). We'll see that certain network functions, such as the translation of human-friendly names for Internet end systems like `www.ietf.org` to a 32-bit network address, are also done with the help of a specific application-layer protocol, namely, the domain name system (DNS). An application-layer protocol is distributed over multiple end systems, with the application in one end system using the protocol to exchange packets of information with the application in another end system. We'll refer to this packet of information at the application layer as a **message**.

The Internet protocol stack consists of 5 layers

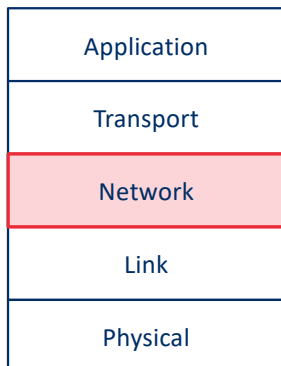


Transport Layer

- Transports application-layer message between end-systems
- Transport Control Protocol (TCP)
 - Connection-oriented
 - Reliable (retransmits lost packets)
 - Flow control (sender/receiver speed matching)
 - Congestion control (throttles transmission rate to avoid network congestion)
- User Datagram Protocol (UDP)
 - Connection-less
 - Unreliable, no flow control, no congestion control
- Transport-layer packets are referred to as **segments**

The Internet's transport layer transports application-layer messages between application endpoints. In the Internet, there are two transport protocols, TCP and UDP, either of which can transport application-layer messages. TCP provides a connection-oriented service to its applications. This service includes guaranteed delivery of application-layer messages to the destination and flow control (that is, sender/receiver speed matching). TCP also breaks long messages into shorter segments and provides a congestion-control mechanism, so that a source throttles its transmission rate when the network is congested. The UDP protocol provides a connectionless service to its applications. This is a no-frills service that provides no reliability, no flow control, and no congestion control. In this book, we'll refer to a transport-layer packet as a **segment**.

The Internet protocol stack consists of 5 layers



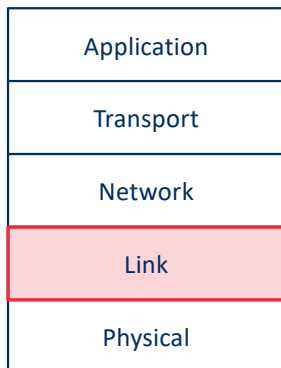
Network Layer

- Responsible for moving network-layer packet from one host to another
- Functionality
 - Addressing (Internet Protocol (IP) versions 4 and 6)
 - Routing (many intra- and inter-domain routing protocols)
- Network-layer packets are called **datagrams**

The Internet's network layer is responsible for moving network-layer packets known as **datagrams** from one host to another. The Internet transport-layer protocol (TCP or UDP) in a source host passes a transport-layer segment and a destination address to the network layer, just as you would give the postal service a letter with a destination address. The network layer then provides the service of delivering the segment to the transport layer in the destination host.

The Internet's network layer includes the celebrated IP protocol, which defines the fields in the datagram as well as how the end systems and routers act on these fields. There is only one IP protocol, and all Internet components that have a network layer must run the IP protocol. The Internet's network layer also contains routing protocols that determine the routes that datagrams take between sources and destinations. The Internet has many routing protocols. The Internet is a network of networks, and within a network, the network administrator can run any routing protocol desired. Although the network layer contains both the IP protocol and numerous routing protocols, it is often simply referred to as the IP layer, reflecting the fact that IP is the glue that binds the Internet together.

The Internet protocol stack consists of 5 layers



Link Layer

- Responsible for transferring data across a single link (i.e., between 2 nodes)
- Functionality depends on the specific link-layer protocol
 - Reliability
 - Error detection
 - Addressing
 - Media access
 - ...
- Examples
 - Ethernet
 - Wi-Fi
 - DOCSIS (Data Over Cable Service Interface Specification)
 - PPP (Point-to-Point Protocol)
- Link-layer packets are called **frames**

The Internet's network layer routes a datagram through a series of routers between the source and destination. To move a packet from one node (host or router) to the next node in the route, the network layer relies on the services of the link layer. In particular, at each node, the network layer passes the datagram down to the link layer, which delivers the datagram to the next node along the route. At this next node, the link layer passes the datagram up to the network layer.

The services provided by the link layer depend on the specific link-layer protocol that is employed over the link. For example, some link-layer protocols provide reliable delivery, from transmitting node, over one link, to receiving node. Note that this reliable delivery service is different from the reliable delivery service of TCP, which provides reliable delivery from one end system to another. Examples of link-layer protocols include Ethernet, WiFi, and the cable access network's DOCSIS protocol. As datagrams typically need to traverse several links to travel from source to destination, a datagram may be handled by different link-layer protocols at different links along its route. For example, a datagram may be handled by Ethernet on one link and by PPP on the next link. The network layer will receive a different service from each of the different link-layer protocols. In this book, we'll refer to the link-layer packets as **frames**.

The Internet protocol stack consists of 5 layers

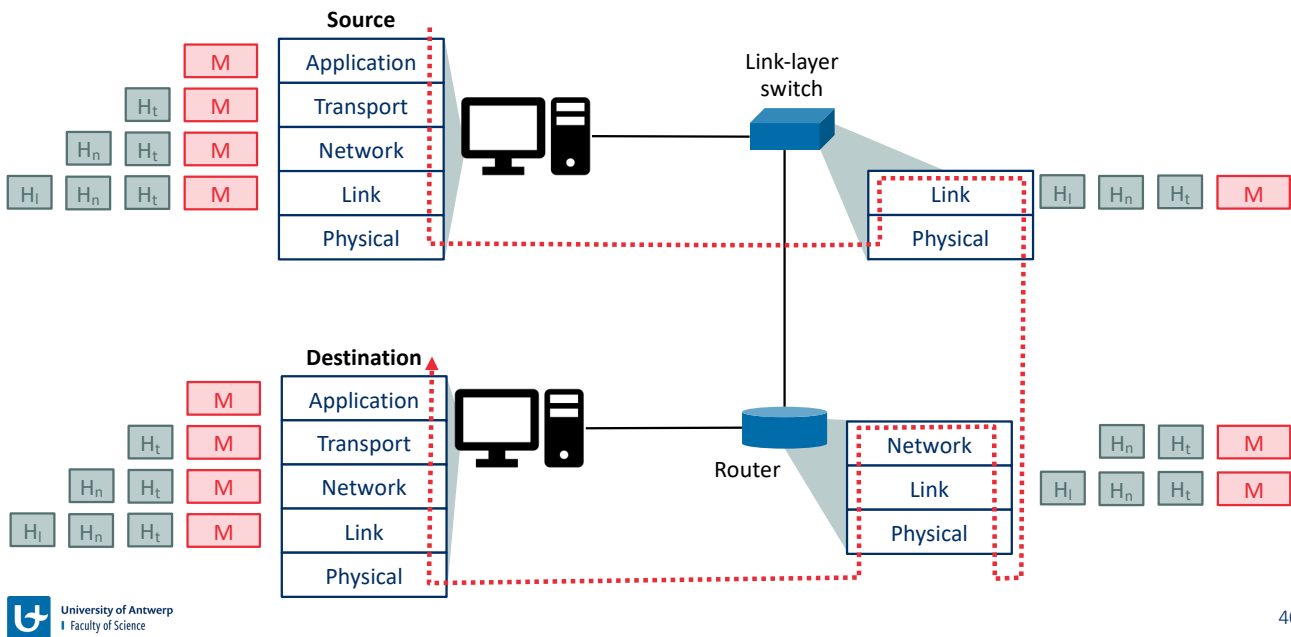
Application
Transport
Network
Link
Physical

Physical Layer

- Responsible for transferring individual symbols (one or more bits) over the physical medium
- Functionality
 - Modulation & coding
 - Bit synchronization
 - ...
- Depends on the type of medium
 - Twisted-pair copper wire
 - Single-mode fiber optics
 - Free-space optical waves
 - Radio frequency waves
 - ...

While the job of the link layer is to move entire frames from one network element to an adjacent network element, the job of the physical layer is to move the *individual bits* within the frame from one node to the next. The protocols in this layer are again link dependent and further depend on the actual transmission medium of the link (for example, twisted-pair copper wire, single-mode fiber optics). For example, Ethernet has many physical-layer protocols: one for twisted-pair copper wire, another for coaxial cable, another for fiber, and so on. In each case, a bit is moved across the link in a different way.

Encapsulation



40

The figure shows the physical path that data takes down a sending end system's protocol stack, up and down the protocol stacks of an intervening link-layer switch and router, and then up the protocol stack at the receiving end system. Routers and link-layer switches are both packet switches. Similar to end systems, routers and link-layer switches organize their networking hardware and software into layers. But routers and link-layer switches do not implement *all* of the layers in the protocol stack; they typically implement only the bottom layers. Link-layer switches implement layers 1 and 2; routers implement layers 1 through 3. This means, for example, that Internet routers are capable of implementing the IP protocol (a layer 3 protocol), while link-layer switches are not. We'll see later that while link-layer switches do not recognize IP addresses, they are capable of recognizing layer 2 addresses, such as Ethernet addresses. Note that hosts implement all five layers; this is consistent with the view that the Internet architecture puts much of its complexity at the edges of the network.

The figure also illustrates the important concept of **encapsulation**. At the sending host, an **application-layer message** (M in the figure) is passed to the transport layer. In the simplest case, the transport layer takes the message and appends additional information (so-called transport-layer header information, H_t) that will be used by the receiver-side transport layer. The application-layer message and the transport-layer header information together constitute the **transport-layer segment**. The transport-layer segment thus encapsulates the application-layer message. The added information might include information allowing the receiver-side transport layer to deliver the message up to the appropriate application, and error-detection bits that allow the receiver to determine whether bits in the message have been changed in route. The transport layer

then passes the segment to the network layer, which adds network-layer header information (H_n) such as source and destination end system addresses, creating a **network-layer datagram**. The datagram is then passed to the link layer, which (of course!) will add its own link-layer header information and create a **link-layer frame**. Thus, we see that at each layer, a packet has two types of fields: header fields and a **payload field**. The payload is typically a packet from the layer above.

Summary

Summary

- The Internet evolved from a network of 4 computers in 1967 to a **network of networks** containing billions of devices
- It consists of a layered hierarchy of **Internet Service Providers** (ISPs), together forming a global network connecting all hosts
- It mostly makes use of **packet switching**, which is more efficient and simple, but less predictable than **circuit switching**
- Network **performance** is characterised in terms of delay, packet loss and throughput
- A **protocol** defines the format and order of messages exchanged between hosts, and actions taken upon receiving/transmitting these messages
- The Internet **protocol stack** consists of 5 layers (application, transport, network, link, and physical), each encapsulating messages with their own headers
- The contents of this part is covered in the **book** in Sections 1.1, 1.2, 1.3, 1.4, 1.5 and 1.7



University of Antwerp
| Faculty of Science

End of Part 1